Grading Rubric — Assignment 4

Campus Bike-Share: Data Structures End to End

Total: 100 points

How to use this rubric

This rubric is the grading authority. There is **no answer key**, and the reference solutions document is one acceptable answer set rather than the standard to match against. Each criterion below says what an answer must *demonstrate*; it does not say what words it must contain.

Three rules apply throughout:

- **Any correct approach earns the marks.** Where a question has several defensible answers, the criterion says so. Do not deduct because a student chose differently from the reference solution.
- **Reasoning is the deliverable on every "justify", "derive" or "compare" part.** A correct conclusion with no argument earns the marks for the conclusion only. A well-argued answer that reaches a different defensible conclusion earns full marks.
- **Code is graded on whether it works and meets the stated complexity.** Do not deduct for naming, formatting, comments, or for not using the library or idiom you would have used. Do deduct for faults that stop it working: wrong operator or direction, off-by-one, undefined name, a method that does not do what the author thinks.

Q1. Choosing the right structure — 12 points

(a) Choice and justification — 5 points

- **5** — Names a structure whose cost profile fits the workload (a hash table/dict and a balanced BST are both defensible) AND justifies it from the access pattern: identifies that lookup dominates by orders of magnitude and that the report is rare, and trades the two against each other explicitly.
- **3** — Names a suitable structure and gives a partial justification: correct complexities, but no engagement with the *relative frequency* of the operations.
- **1** — Names a suitable structure with no justification, or a justification that is generic ("hash tables are fast").
- **0** — Names a structure whose cost profile does not fit (e.g. a plain list or linked list for 40,000 lookups) without acknowledging the cost.

(b) Effect of hourly reporting — 4 points

- **4** — Reaches a conclusion (either "no change" or "switch") and supports it with a **quantitative** comparison: estimates the cost of the report against the cost of the lookups at their stated frequencies.
- **2** — Reaches a defensible conclusion with a qualitative argument only.
- **0** — Answers yes or no without comparing costs.

Both conclusions are correct if argued. Do not reward the reference answer's conclusion over the other one.

(c) Complexities — 3 points

- **3** — All four operations correct and consistent with the structure the student chose in (a) — including the report, whose cost differs by structure (O(n log n) for a hash table, O(n) for a tree).
- **2** — Three of four correct, or all four correct for a *different* structure than the one chosen in (a).
- **1** — Two correct.
- **0** — Fewer than two correct.

Average case was asked for. Do not deduct for also giving worst case.

Q2. Hashing — 14 points

(a) Chaining vs open addressing — 4 points

- **2 pts** — The mechanism of both is described correctly: chaining stores collisions in a secondary structure at the slot; open addressing stores all entries in the table and probes for a free slot.
- **1 pt** — A genuine advantage of chaining (tolerates alpha > 1, simple deletion, graceful degradation).
- **1 pt** — A genuine advantage of open addressing (no pointer overhead, cache locality/contiguous memory).

An "advantage" that is just the restatement of the mechanism earns nothing.

(b) Load factor — 3 points

- **1 pt** — alpha = 0.75.
- **2 pts** — Explains that expected probe length grows without bound as alpha -> 1 AND gives the mechanism: **primary clustering** — occupied slots merge into runs, any key hashing into a run must traverse it, and each insertion extends the run. Quoting the formula without the mechanism earns 1 of these 2.

(c) Deletion under open addressing — 3 points

- **2 pts** — Identifies that emptying a slot **breaks probe chains passing through it**, so keys stored beyond the hole become unfindable.
- **1 pt** — Names tombstones / deleted markers as the fix AND states a cost (they count toward the load factor, accumulate, require periodic rehashing).

A student who says only "you must rehash the table" has a working fix and earns the 1 point for it.

(d) Linear probing trace — 4 points

Correct final table:

```
index:  0     1     2     3     4     5     6
value:  -    15     8    22    11     -    27
```

- **3 pts** — Final table exactly correct. Award 3 even if the working is not shown.
- **1 pt** — The probe sequences for 8 (1 -> 2) and 22 (1 -> 2 -> 3) are shown.
- Partial: if the table is wrong, award **1** if at least three slots are right and the probing method is visibly correct — this part tests the procedure, so an arithmetic slip should not cost all four marks.

Q3. The k nearest stations — 18 points

(a) Implementation — 10 points

- **6 pts** — **Correctness.** Returns the k nearest station ids, nearest first. Trace the logic; run it if you can. Deduct in full only for faults that break it (wrong comparison direction, using a min-heap where a max-heap is needed without compensating, returning objects where ids were asked for).
- **3 pts** — **Meets O(n log k).** The heap is bounded at k — the student evicts rather than accumulating. An O(n log n) solution that sorts everything is correct but does not meet the stated bound: award 0 of these 3, and full credit for the other criteria it satisfies.
- **1 pt** — **Edge cases.** `k <= 0` returns empty, and `k > n` does not crash.

Full credit regardless of approach: a bounded max-heap, `heapq.nsmallest`*, or quickselect all qualify if the complexity bound holds. Squared distance instead of* `sqrt` *is a nice optimisation but is* ***not*** *required and earns no extra marks — nor does its absence cost any.*

(b) Complexity justification — 4 points

- **4** — States O(n log k) and accounts for it: n elements examined, each costing at most one O(log k) heap operation **because the heap is capped at k**. The causal link between the cap and the log k is what is being marked.
- **2** — States O(n log k) with a partial account, or accounts correctly for a different (correct) complexity matching their own code.
- **0** — States a complexity with no derivation, or one that does not match the code they wrote.

If the student's code is O(n log n), the correct answer here is O(n log n). Mark the consistency of the analysis with their own code.

(c) Heap vs sorting — 4 points

- **2 pts** — Explains the saving concretely for n = 40,000, k = 5: log k ~ 2.3 against log n ~ 15.3, and/or O(k) space against O(n). Numbers or a clear proportional argument both count.
- **2 pts** — Identifies that the advantage vanishes as **k approaches n** (at k = n the two are asymptotically equal). A student who argues sorting wins earlier in practice, on constant factors, earns these 2 as well.

Q4. Shortest walking route — 20 points

(a) Implementation — 10 points

- **5 pts** — **Dijkstra is correct.** Maintains tentative distances, always settles the minimum unsettled vertex, relaxes edges from it.
- **3 pts** — **Returns the path, not only the distance**, via predecessor tracking or an equivalent, reconstructed in the right order (source -> target).
- **2 pts** — **Handles the unreachable case** and returns the stated `(inf, [])` shape, or an equivalent the student documents.

Both standard heap patterns are fully correct: pushing duplicate entries and skipping stale pops, or maintaining a visited set with decrease-key. A student who uses neither and re-scans for the minimum has written the O(V²) version — that is a correct algorithm, so award these 10 marks in full and mark the complexity in (b) against what they wrote.

(b) Complexity derivation — 6 points

- **2 pts** — V pops from the heap, each O(log V).
- **2 pts** — E relaxations, each pushing at most once, O(log V) per push.
- **2 pts** — Combines to O((V + E) log V) **and the derivation matches the student's own implementation**.

The question said "do not simply state the result", so a bare "O(E log V)" with no working earns 0 even though it is the right answer. A correct derivation of O(V²) for an array-based implementation, or O(V log V + E) for a Fibonacci heap, earns full marks if that is what they wrote.

(c) Negative weights — 4 points

- **2 pts** — A concrete three-node counterexample with edges and weights given, where a negative edge makes a longer-looking route cheaper.
- **1 pt** — States both what Dijkstra returns and the true shortest distance, and they are correct for the graph the student gave.
- **1 pt** — Names the violated assumption: once a vertex is settled its distance is treated as final, which only holds if extending a path cannot reduce its cost.

Check the student's own graph for internal consistency rather than comparing it to the reference example. Many valid counterexamples exist.

Q5. Amortized analysis — 12 points

(a) Doubling — 6 points

- **2 pts** — Writes the total copy cost as the geometric series 1 + 2 + 4 + … + 2^k (or equivalent).
- **2 pts** — Evaluates it and shows the bound — the sum is < 2n, so the total work over n appends is O(n).
- **2 pts** — Concludes amortized O(1) per append, dividing total by n.

Accept the accounting or potential method instead of the aggregate method if correctly carried out, even though the question named the aggregate method.

(b) Fixed +10 growth — 6 points

- **3 pts** — Derives the total copy cost as the arithmetic series 10 + 20 + … + n ~ $n^2/20$, i.e. Theta(n^2). The derivation must be present; the bare answer earns 1 of these 3.
- **1 pt** — States the resulting amortized cost per append: Theta(n).
- **2 pts** — Judges the trade and supports it: the memory saved is a constant factor while the time lost is a factor of n, so the proposal is bad. A student who identifies the general principle — geometric growth gives O(1), arithmetic growth gives O(n) — earns these 2 outright.

Q6. Debugging — 12 points

(a) Both faults — 6 points

- **3 pts** — **Fault 1:** `hi = len(sorted_ids)` is a half-open bound used with the inclusive condition `lo <= hi`, allowing `mid` to reach `len` and index out of range.
- **3 pts** — **Fault 2:** `lo = mid` fails to exclude the midpoint, so once `hi == lo + 1` the interval stops shrinking and the loop does not terminate.

Award per fault. Identifying the line without explaining the consequence earns 1 of the 3 for that fault. A student who reports a third "fault" that is not one loses nothing, provided both real faults are among their answers.

(b) Masking and triggering inputs — 4 points

- **2 pts** — Identifies that **fault 1 is hidden by fault 2** and explains why: `lo` advances only via `lo = mid`, which cannot reach `len`, so the loop hangs before any out-of-range index is reached. No input exposes fault 1 while fault 2 is present.
- **1 pt** — A concrete input demonstrating the observable failure — any absent target above the first element, e.g. `[10, 20, 30]` with target 25 — and states that it hangs.
- **1 pt** — A concrete input that exposes fault 1 once fault 2 is repaired — any absent target greater than every element, e.g. `[10, 20, 30]` with target 35 — and states that it raises `IndexError`.

This is the hard part of the question and most students will miss the masking. A student who gives two plausible-but-wrong trigger inputs, one per fault, without noticing the masking, earns at most the 1 point for the hang.

(c) Corrected function — 2 points

- **2** — Both faults fixed and the result is a correct binary search. The inclusive form (`hi = len - 1`, `while lo <= hi`, `hi = mid - 1`) and the half-open form (`hi = len`, `while lo < hi`, `hi = mid`) are equally correct.
- **1** — One fault fixed.
- **0** — Neither fixed, or the "fix" introduces a new fault.

Q7. Designing a spatial query — 12 points

No recommendation is required to be correct. The marks are for the comparison and the argument. A well-argued case for the linear scan and a well-argued case for a spatial index both earn 12.

Comparison of two approaches — 8 points

Award per approach (4 points each), one of which must be the linear scan:

- **2 pts** — Query complexity stated and correct for that approach, with what it depends on (for a grid: the number of cells overlapping the radius and the bikes within them, not simply "O(1)").
- **1 pt** — Cost of a bike moving, correctly identified (free for the scan; constant-time bucket maintenance for an index).
- **1 pt** — Memory overhead, correctly identified.

Recommendation — 4 points

- **4** — Recommends one approach and argues it **against this specific workload** — engaging with at least two of: the 2 km radius against a 5 km campus, 500 queries/second, 40,000 objects, constant movement.
- **2** — Recommends one approach with a general argument that does not engage with the specific numbers.
- **0** — States a preference with no argument, or recommends nothing.

The strongest available insight is that a 2 km radius on a 5 km campus returns roughly half the data, and an index cannot prune what the query selects. Award the full 4 for spotting it — but do not require it: an argument from query throughput, from growth headroom, or from the cost of motion is equally valid, and a k-d tree correctly rejected for rebalancing badly under constant movement is also a full-credit line of reasoning.